NAME:RAUNAK SOLOMON BAILEY
UNIVERSITY ROLL NO: 2025611
SECTION: B

QUES14 Build a Library Management System . The system needs to handle Books, Magazines, and E-books, each with behaviors such as borrowing, returning, and calculating late fees. You need to design an interface that defines common behaviors for all types of items, such as borrowing, returning, and calculating late fees. You will also define specific methods that each type of item must implement, as well as default methods that provide common behavior shared across different item types. Additionally, the system should handle exceptions such as invalid return dates and overdue items.
CODE:

```
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import java.util.Scanner;
class InvalidItemException extends Exception {
   public InvalidItemException(String msg) {
      super(msg);
   }
}
interface LibraryOperations {
   void borrowItem(LocalDate borrowDate);
   void returnItem(LocalDate returnDate);
   double calculateLateFee(LocalDate returnDate);
   static boolean isValidItemId(String itemId) {
      return itemId.matches("ITEM\\d{4}");
   }
}
class Book implements LibraryOperations {
   String itemId, title;
   boolean borrowed = false;
   LocalDate dueDate;
   public Book(String id, String title) throws InvalidItemException {
      if (!LibraryOperations.isValidItemId(id)) {
         throw new InvalidItemException("Invalid ID!");
      }
      this.itemId = id;
      this.title = title;
   }
   public void borrowItem(LocalDate borrowDate) {
      borrowed = true;
      dueDate = borrowDate.plusDays(7);
      System.out.println("Borrowed! Due Date: " + dueDate);
   }
   public void returnItem(LocalDate returnDate) {
      double fine = calculateLateFee(returnDate);
```

```
        if (fine > 0)
            System.out.println("Late! Fine = ₹" + fine);
        else
            System.out.println("Returned on time");
        borrowed = false;
}
    public double calculateLateFee(LocalDate returnDate) {
        if (returnDate.isAfter(dueDate)) {
            long days = ChronoUnit.DAYS.between(dueDate, returnDate);
            return days * 10;
        }
        return 0;
    }
}
 class Magazine implements LibraryOperations {
                    String itemId, title;
   LocalDate dueDate;
    public Magazine(String id, String title) throws InvalidItemException {
        if (!LibraryOperations.isValidItemId(id)) {
            throw new InvalidItemException("Invalid ID!");
        }
        this.itemId = id;
        this.title=title;
    }
    public void borrowItem(LocalDate borrowDate) {
        dueDate = borrowDate.plusDays(5);
        System.out.println("Magazine Borrowed! Due: " + dueDate);
    }
    public void returnItem(LocalDate returnDate) {
        double fine=calculateLateFee(returnDate);
        System.out.println(fine > 0 ? "Fine = ₹" + fine : "Returned on time");
    }
    public double calculateLateFee(LocalDate returnDate) {
        if (returnDate.isAfter(dueDate)) {
            long days = ChronoUnit.DAYS.between(dueDate, returnDate);
            return days * 5;
        }
        return 0;
    }
}
class EBook implements LibraryOperations {
    String itemId, title;
    public EBook(String id, String title) throws InvalidItemException {
        if (!LibraryOperations.isValidItemId(id)) {
            throw new InvalidItemException("Invalid ID!");
        }
```

```
        this.itemId = id;
        this.title=title;
    }
    public void borrowItem(LocalDate borrowDate) {
        System.out.println("EBook Downloaded!");
    }
    public void returnItem(LocalDate returnDate) {
        System.out.println("No return needed");
    }
    public double calculateLateFee(LocalDate returnDate) {
        return 0;
    }}
public class fourteen{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
     try {System.out.print("Enter Item ID (ITEM1234): ");
            String id = sc.next();
            System.out.print("Enter Title: ");
            String title = sc.next();
            System.out.println("1.Book 2.Magazine 3.EBook");
            int type = sc.nextInt();
            LibraryOperations item;
            if (type == 1)
                item = new Book(id, title);
            else if (type == 2)
                item = new Magazine(id, title);
            else
                item = new EBook(id, title);
            int choice;
            do {
                System.out.println("\n1.Borrow 2.Return 3.Exit");
                choice = sc.nextInt();
                switch (choice) {
            case 1: System.out.print("Enter Borrow Date (YYYY-MM-DD): ");
                        LocalDate bDate = LocalDate.parse(sc.next());
                        item.borrowItem(bDate);
                        break;
            case 2: System.out.print("Enter Return Date (YYYY-MM-DD): ");
                        LocalDate rDate = LocalDate.parse(sc.next());
                        item.returnItem(rDate);
                        break;}
            } while (choice != 3);
        } catch (InvalidItemException e) {
            System.out.println("Error: " + e.getMessage());
        }
        sc.close();}}
```

OUTPUT

```
C:\Users\VIVEK\OneDrive\Desktop>javac fourteen.java

C:\Users\VIVEK\OneDrive\Desktop>java fourteen
Enter Item ID (ITEM1234): ITEM2024
Enter Title: CSE
1.Book 2.Magazine 3.EBook
1

1.Borrow 2.Return 3.Exit
1
Enter Borrow Date (YYYY-MM-DD): 2026-03-05
Borrowed! Due Date: 2026-03-12

1.Borrow 2.Return 3.Exit
2
Enter Return Date (YYYY-MM-DD): 2026-03-25
Late Fine = 130.0

1.Borrow 2.Return 3.Exit
3
```

NAME: VIVEK MAINALI
UNIVERSITY ROLL NO: 2025592
SECTION: B

QUES15 Design a Java application to validate student admission details using custom exceptions. The system checks that:

- **Name is not empty**
- **Age is between 17–25**
- **Email contains '@' and a valid domain**
- **Marks are between 0–100**
- **Marks≥ 60 are required for admission**

CODE:

```
import java.util.Scanner;
class InvalidNameException extends Exception {
   public InvalidNameException(String msg) { super(msg); }
}
class InvalidAgeException extends Exception {
   public InvalidAgeException(String msg) { super(msg); }
}
class InvalidEmailException extends Exception {
   public InvalidEmailException(String msg) { super(msg); }
}
class InvalidMarksException extends Exception {
   public InvalidMarksException(String msg) { super(msg); }
}
class NotEligibleException extends Exception {
   public NotEligibleException(String msg) { super(msg); }
}
class Student {
   String name, email;
   int age;
   double marks;
   public Student(String name, int age, String email, double marks)
         throws InvalidNameException, InvalidAgeException,
         InvalidEmailException, InvalidMarksException, NotEligibleException {
      if (name == null || name.trim().isEmpty())
         throw new InvalidNameException("Name cannot be empty");
      if (age < 17 || age > 25)
         throw new InvalidAgeException("Age must be 17–25");
      if (!email.contains("@") || !email.contains("."))
         throw new InvalidEmailException("Invalid Email");
      if (marks < 0 || marks > 100)
         throw new InvalidMarksException("Marks must be 0–100");
      if (marks < 60)
         throw new NotEligibleException("Marks < 60 → Not Eligible");
      this.name = name;
```

```
        this.age = age;
        this.email = email;
        this.marks = marks;
    }
    public void display() {
        System.out.println("Name: " + name + ", Age: " + age +
                ", Email: " + email + ", Marks: " + marks);
    }
}
public class fifteen {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Student[] students = new Student[5]; // fixed size array
        int count = 0;
        while (count < students.length) {
            try {
                System.out.println("\nEnter Student Details:");
                System.out.print("Name: ");
                String name = sc.nextLine();
                System.out.print("Age: ");
                int age = Integer.parseInt(sc.nextLine());
                System.out.print("Email: ");
                 String email = sc.nextLine();
                  System.out.print("Marks: ");
                double marks = Double.parseDouble(sc.nextLine());
                students[count] = new Student(name, age, email, marks);
                count++;
                System.out.println("Admission Successful!");
            } catch (Exception e) {
                System.out.println("Error:  "  +  e.getMessage());
                System.out.println("Please re-enter details.\n");
            }
        }
System.out.println("\n--- Admitted Students ---");
        for (int i = 0; i < count; i++) {
            students[i].display();
        }
        sc.close();
    }
```

OUTPUT

```
C:\Users\VIVEK\OneDrive\Desktop>javac fifteen.java

C:\Users\VIVEK\OneDrive\Desktop>javafi   een
Enter Student Details:
Name: Vivek Mainali
Age: 19
Email: Vm8111@gmail.com
Marks: 98
Admission Successful!

Enter Student Details:
Name: Ansh
Age: 19
Email: sdsd
Marks: 87
Error: Invalid Email
Please re-enter details.

Enter Student Details:
Name: Riya
Age: 16
Email: riya@gmail.com
Marks: 75
Error: Age must be 17–25
Please re-enter details.


Enter Student Details:
Name: Riya Sharma
Age: 20
Email: riya@gmail.com
Marks: 55
Error: Marks < 60 → Not Eligible
Please re-enter details.


--- Admitted Students ---
Name: Vivek Mainali, Age: 19, Email: Vm8111@gmail.com, Marks: 98.0
```

NAME: RAUNAK
SOLOMON BAILEY **UNIVERSITY ROLL NO:** 2025611 **SECTION:** B

<u>QUES16</u> Develop a Java application that reads a text file (input.txt), processes each line by performing string operations, and writes the results to an output file (output.txt).
Operations include:
- **Converting text to uppercase.**
- **Counting words and vowels (A, E, I, O, U).**
- **Replacing multiple spaces with single spaces**
- **Reversing each word (but keeping word order)**

<u>CODE:</u>

```
import java.util.*;
importjava.io.*;

public class SimpleTextProcessor {
   public static void main(String[] args) {
      int totalLines = 0;
      int totalWord = 0;

      int a = 0, e = 0, i = 0, o = 0, u = 0;

      try {
         BufferedReader br = new BufferedReader(new FileReader("input.txt"));
         BufferedWriter bw = new BufferedWriter(new FileWriter("output.txt"));

         String line;
         int lineNo = 1;

         while ((line = br.readLine()) != null) {
            totalLines++;
            line = line.trim().replaceAll("\\s+", " ");
            String upper = line.toUpperCase();

            String words[] = line.split(" ");
            int wordCount = (line.isEmpty()) ? 0 : words.length;
            totalWords += wordCount;

            for (char ch : upper.toCharArray()) {
               if (ch == 'A') a++;
                else if (ch == 'E') e++;
                  else if (ch == 'I') i++;
               else if (ch == 'O') o++;
               else if (ch == 'U') u++;
            }

            String reversedLine = "";
```

```
            for (String w : words) {
                String rev = "";
                for (int j = w.length() - 1; j >= 0; j--) {
                    rev += w.charAt(j);
                }
                reversedLine += rev + " ";
            }

            bw.write("Line " + lineNo + "\n");
            bw.write("Uppercase: " + upper + "\n");
            bw.write("Words: " + wordCount + "\n");
            bw.write("Reversed: " + reversedLine.trim() + "\n");
            bw.write("----------------------\n");

            lineNo++;
        }

        br.close();
        bw.close();

    } catch (Exception
e1) {
        System.out.println("Error: " + e1.getMessage());
    }

    System.out.println("Total Lines: " + totalLines);
    System.out.println("Total Words: " + totalWords);

    System.out.println("Vowels Count:");
    System.out.println("A: " + a);
     System.out.println("E: " + e);
       System.out.println("I:" + i);
    System.out.println("O:  "  +  o);
    System.out.println("U: " + u);
  }
}
```

OUTPUT

input.txt

```
hello   world
java is   fun
I love programming
madam
racecar
this    is    a   test
```

output.txt

```
Line 1
Uppercase: HELLO WORLD
Words: 2
Reversed: olleh dlrow
----------------------
Line 2
Uppercase: JAVA IS FUN
Words: 3
Reversed: avaj si nuf
----------------------
Line 3
Uppercase: I LOVE PROGRAMMING
Words: 3
Reversed: I evol gnimmargorp
----------------------
Line 4
Uppercase: MADAM
Words: 1
Reversed: madam
----------------------
Line 5
Uppercase: RACECAR
Words: 1
Reversed: racecar
----------------------
Line 6
Uppercase: THIS IS A TEST
Words: 4
Reversed: siht si a tset
----------------------
```

C:\Users\VIVEK\OneDrive\Desktop>javac SimpleTextProcessor.java

C:\Users\VIVEK\OneDrive\Desktop>java SimpleTextProcessor
TotalLines:6
Total Words: 14
VowelsCount:
A: 8
E:4
I: 5
O: 4
U: 1